

Lab 3: SQL数据处理和分析实习

任务成员

毛川2300013218

关睿轩2300013238

芮思铭2300013094

Task 1: SQL数据预处理实习任务

1. 环境准备与数据库创建

首先导入必要的库并创建SQLite数据库连接。

```
In [1]: import sqlite3
import pandas as pd
import numpy as np

conn = sqlite3.connect('user_cleaning.db')
cursor = conn.cursor()
```

2. 创建原始数据表

创建包含脏数据的用户原始数据表，模拟真实业务场景中的各种数据质量问题。

```
In [2]: cursor.execute('DROP TABLE IF EXISTS user_raw_data')

cursor.execute('''
CREATE TABLE user_raw_data (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    raw_name TEXT,           -- 原始姓名（含空格、大小写混乱）
    raw_phone TEXT,          -- 原始手机号（含符号、空格、长度错误）
    raw_email TEXT,          -- 原始邮箱（格式不规范、冗余字符）
    raw_register_time TEXT,  -- 原始注册时间（格式混乱）
    raw_address TEXT,        -- 原始地址（含冗余字符、空格）
    raw_age TEXT,            -- 原始年龄（含文字、符号、空值）
    raw_remark TEXT          -- 备注（含特殊字符、空值）
)
''')
```

```
Out[2]: <sqlite3.Cursor at 0x1e82b5975c0>
```

3. 插入脏测试数据

插入包含各种脏数据的测试数据，覆盖所有需要清洗的场景：

- 姓名：前后空格、中间空格、英文大小写混乱
- 手机号：含横杠、空格、字母、86前缀、长度不足
- 邮箱：缺少@、双@、冗余字符、大小写混乱
- 注册时间：多种日期格式混搭、异常日期
- 地址：含前后空格
- 年龄：含'岁'字、'未知'、空值
- 备注：含空格、NULL值、空字符串

In [3]: # 插入脏测试数据

```
raw_data = [
    ('张三', '13800138000', 'zhangsan@163.com', '2024-01-15 09:30:00', '北京市朝',
     '李四', '13912345678', 'lisi@gmail.com', '2024/02/20', '上海市浦东新区张江',
     '王五', '137-8888-9999', 'wangwu@qq..com', '2024.03.10', '广州市天河区珠江',
     'Zhao Liu', '13677778888', 'zhaoliu123', '2024-04-05', '深圳市南山区科技园',
     '钱七', '861356667777', 'qianqi@outlook.com', '2024/05/12 10:00', '杭州市西',
     '孙八', '1345555', 'sunba@126.com', '2024.06.18 14:20', '成都市高新区天府大',
     '周九', '133abc123456', 'zhoujiu@', '2024-07-22', '武汉市洪山区光谷广场',
     '吴十', '13211112222', 'WU_SHI@qq.com', '2024/08/30', '重庆市渝中区解放碑',
     '郑十一', '13100001111', 'zhengeshiyi123@', '2024.09.05', '西安市雁塔区高新',
     '王十二', '13099998888', 'wangshier@163.com', '2024年10月01日', '长沙市岳麓',
     '李十三', '12988887777', 'lisan@qq.com', '2024-13-01', '青岛市市南区香港中路',
     ('张十四', '', 'zhangsi@126.com', '', '昆明市五华区南屏街', '未知', ''))
]

cursor.executemany('''
INSERT INTO user_raw_data
(raw_name, raw_phone, raw_email, raw_register_time,
 raw_address, raw_age, raw_remark)
VALUES (?, ?, ?, ?, ?, ?, ?)
''', raw_data)

conn.commit()
```

4. 查看原始数据

查看插入的原始数据，了解脏数据的实际情况。

In [4]: print('原始数据')

```
raw_df = pd.read_sql_query(
    'SELECT * FROM user_raw_data',
    conn
)
raw_df
```

原始数据

Out[4]:

	id	raw_name	raw_phone	raw_email	raw_register_time	raw_address
0	1	张三	13800138000	zhangsan@163.com	2024-01-15 09:30:00	北京市昌平区建国路
1	2	李四	13912345678	lisi@gmail.com	2024/02/20	上海市浦东新区张江路
2	3	王五	137-8888-9999	wangwu@qq.com	2024.03.10	广州市天河区珠江
3	4	Zhao Liu	13677778888	zhaoliu123	2024-04-05	深圳市南山区
4	5	钱七	8613566667777	qianqi@outlook.com	2024/05/12 10:00	杭州市西湖区
5	6	孙八	1345555	sunba@126.com	2024.06.18 14:20	成都市高新区天府
6	7	周九	133abc123456	zhoujiu@	2024-07-22	武汉市洪山区光谷
7	8	吴十	13211112222	WU_SHI@@qq.com	2024/08/30	重庆市南岸区
8	9	郑十一	13100001111	zhengeshiyi123@	2024.09.05	西安市雁塔区
9	10	王十二	13099998888	wangshier@163.com	2024年10月01日	长沙市岳麓区
10	11	李十三	12988887777	lisan@qq.com	2024-13-01	青岛市市南区香港
11	12	张十四		zhangsi@126.com		昆明市五华区

5. 数据清洗与标准化

清洗规则说明

- 1. **姓名标准化**: 去除姓名中的前后空格和中间多余空格
- 2. **手机号清洗**: 去除横杠、空格、字母等冗余字符，只保留11位数字
- 3. **邮箱标准化**: 转换为小写，校验格式 (必须包含@和.)
- 4. **日期标准化**: 将各种日期格式统一转换为YYYY-MM-DD格式
- 5. **地址标准化**: 去除地址中的前后空格
- 6. **年龄标准化**: 提取纯数字年龄，非数字标记为"未知年龄"
- 7. **备注标准化**: 空值、NULL、空格统一替换为"无备注"

新增清洗任务

- 8. **姓名长度校验**: 检查姓名长度是否合法 (≥2个字符)
- 9. **手机号运营商识别**: 根据手机号前三位识别运营商
- 10. **邮箱服务商提取**: 识别QQ邮箱、网易邮箱、Gmail等

11. **用户年龄分类**: 将用户分为青年、中年等年龄段

12. **地址省份提取**: 从地址中提取省份信息

```
In [6]: # 创建标准化数据表
cursor.execute('DROP TABLE IF EXISTS user_standard_data')

# SQLite的REGEXP函数需要手动启用, 这里使用LIKE和GLOB替代
# 注意: SQLite中默认没有REGEXP_REPLACE函数, 这里使用嵌套REPLACE处理

cursor.execute('''
CREATE TABLE user_standard_data AS
SELECT
    id,

    -- 任务1: 姓名标准化 (去除空格)
    TRIM(REPLACE(raw_name, ' ', '')) AS user_name,

    -- 任务2: 手机号清洗
    CASE
        WHEN LENGTH(
            REPLACE(
                REPLACE(
                    REPLACE(
                        REPLACE(raw_phone, '-', ''),
                        ' ', ''),
                    'abc', ''),
                '86', '')
            ) = 11
        THEN REPLACE(
            REPLACE(
                REPLACE(
                    REPLACE(raw_phone, '-', ''),
                    ' ', ''),
                'abc', ''),
            '86', '')
        ELSE '无效手机号'
    END AS user_phone,

    -- 任务3: 邮箱标准化
    CASE
        WHEN raw_email LIKE '%@%.'
            AND raw_email NOT LIKE '%@%'
            AND raw_email NOT LIKE '%.%'
            AND raw_email NOT LIKE '%@%@%'
        THEN LOWER(raw_email)
        ELSE '无效邮箱'
    END AS user_email,

    -- 任务4: 日期标准化
    CASE
        WHEN raw_register_time != '' AND raw_register_time IS NOT NULL
        THEN SUBSTR(
            REPLACE(
                REPLACE(
                    REPLACE(
                        REPLACE(raw_register_time, '年', '-'),
                        '月', '-'),
                    '日', ''),
                '/', '-'),
            1, 10)
    END AS user_register_time
''')
```

```

        1, 10)
    ELSE '无效日期'
END AS register_date,

-- 任务5: 地址标准化
TRIM(raw_address) AS user_address,

-- 任务6: 年龄标准化
CASE
    WHEN raw_age IN ('未知', '', NULL) OR raw_age IS NULL
    THEN '未知年龄'
    ELSE REPLACE(raw_age, '岁', '')
END AS user_age,

-- 任务7: 备注标准化
CASE
    WHEN TRIM(raw_remark) = ''
        OR TRIM(raw_remark) = 'NULL'
        OR raw_remark IS NULL
    THEN '无备注'
    ELSE TRIM(raw_remark)
END AS user_remark,

-- =====
-- 新增任务1: 姓名长度校验
-- =====
CASE
    WHEN LENGTH(TRIM(REPLACE(raw_name, ' ', ''))) >= 2
    THEN '姓名合法'
    ELSE '姓名异常'
END AS name_status,

-- =====
-- 新增任务2: 手机号运营商识别
-- =====
CASE
    WHEN raw_phone LIKE '138%' OR raw_phone LIKE '139%'
        OR raw_phone LIKE '188%' OR raw_phone LIKE '187%'
        OR raw_phone LIKE '135%' OR raw_phone LIKE '136%'
        OR raw_phone LIKE '137%' OR raw_phone LIKE '150%'
        OR raw_phone LIKE '151%' OR raw_phone LIKE '152%'
    THEN '中国移动'
    WHEN raw_phone LIKE '130%' OR raw_phone LIKE '131%'
        OR raw_phone LIKE '132%' OR raw_phone LIKE '155%'
        OR raw_phone LIKE '156%' OR raw_phone LIKE '185%'
        OR raw_phone LIKE '186%'
    THEN '中国联通'
    WHEN raw_phone LIKE '133%' OR raw_phone LIKE '153%'
        OR raw_phone LIKE '180%' OR raw_phone LIKE '181%'
        OR raw_phone LIKE '189%'
    THEN '中国电信'
    ELSE '未知运营商'
END AS operator_type,

-- =====
-- 新增任务3: 邮箱服务商提取
-- =====
CASE
    WHEN raw_email LIKE '%qq.com%' THEN 'QQ邮箱'
    WHEN raw_email LIKE '%163.com%' THEN '网易邮箱'

```

```

        WHEN raw_email LIKE '%126.com%' THEN '网易邮箱'
        WHEN raw_email LIKE '%gmail.com%' THEN 'Gmail邮箱'
        WHEN raw_email LIKE '%outlook.com%' THEN 'Outlook邮箱'
        ELSE '其他邮箱'
    END AS email_provider,

-- =====
-- 新增任务4：用户年龄分类
-- =====
CASE
    WHEN REPLACE(raw_age, '岁', '') GLOB '[0-9]*'
        AND CAST(REPLACE(raw_age, '岁', '') AS INTEGER) < 30
    THEN '青年'
    WHEN REPLACE(raw_age, '岁', '') GLOB '[0-9]*'
        AND CAST(REPLACE(raw_age, '岁', '') AS INTEGER) BETWEEN 30 AND 40
    THEN '中年'
    WHEN REPLACE(raw_age, '岁', '') GLOB '[0-9]*'
        AND CAST(REPLACE(raw_age, '岁', '') AS INTEGER) > 40
    THEN '中老年'
    ELSE '未知'
END AS age_group,

-- =====
-- 新增任务5：地址省份提取
-- =====
CASE
    WHEN raw_address LIKE '北京市%' THEN '北京'
    WHEN raw_address LIKE '上海市%' THEN '上海'
    WHEN raw_address LIKE '广州市%' OR raw_address LIKE '深圳市%'
        OR raw_address LIKE '广东省%'
    THEN '广东'
    WHEN raw_address LIKE '杭州市%' OR raw_address LIKE '浙江省%'
    THEN '浙江'
    WHEN raw_address LIKE '成都市%' OR raw_address LIKE '四川省%'
    THEN '四川'
    WHEN raw_address LIKE '武汉市%' OR raw_address LIKE '湖北省%'
    THEN '湖北'
    WHEN raw_address LIKE '重庆市%' THEN '重庆'
    WHEN raw_address LIKE '西安市%' OR raw_address LIKE '陕西省%'
    THEN '陕西'
    WHEN raw_address LIKE '长沙市%' OR raw_address LIKE '湖南省%'
    THEN '湖南'
    WHEN raw_address LIKE '青岛市%' OR raw_address LIKE '山东省%'
    THEN '山东'
    WHEN raw_address LIKE '昆明市%' OR raw_address LIKE '云南省%'
    THEN '云南'
    ELSE '其他地区'
END AS province

FROM user_raw_data
'''

conn.commit()

```

标准化数据表创建成功！

6. 查看清洗后的数据

查看经过清洗和标准化处理后的数据，对比原始数据可以看到明显的改善。

```
In [7]: print('\n')
print('清洗后的标准化数据')

clean_df = pd.read_sql_query(
    'SELECT * FROM user_standard_data',
    conn
)

clean_df
```

=====
清洗后的标准化数据
=====

Out[7]:

	id	user_name	user_phone	user_email	register_date	user_address
0	1	张三	13800138000	zhangsan@163.com	2024-01-15	北京市朝阳区 建国路88号
1	2	李四	13912345678	lisi@gmail.com	2024-02-20	上海市浦东新区 张江高科技 园区
2	3	王五	13788889999	无效邮箱	2024.03.10	广州市天河区 珠江新城
3	4	ZhaoLiu	13677778888	无效邮箱	2024-04-05	深圳市南山区 科技园
4	5	钱七	13566667777	qianqi@outlook.com	2024-05-12	杭州市西湖区 西溪湿地
5	6	孙八	无效手机号	sunba@126.com	2024.06.18	成都市高新区 天府大道
6	7	周九	无效手机号	无效邮箱	2024-07-22	武汉市洪山区 光谷广场
7	8	吴十	13211112222	无效邮箱	2024-08-30	重庆市渝中区 解放碑
8	9	郑十一	13100001111	无效邮箱	2024.09.05	西安市雁塔区 高新区
9	10	王十二	13099998888	wangshier@163.com	2024-10-01	长沙市岳麓区 大学城
10	11	李十三	12988887777	lisan@qq.com	2024-13-01	青岛市市南区 香港中路
11	12	张十四	无效手机号	zhangsi@126.com	无效日期	昆明市五华区 南屏街



7. 数据质量评估

对清洗前后的数据进行质量对比评估，包括：

- 缺失值统计
- 无效数据统计
- 数据完整性分析

```
In [8]: print('\n')
print('数据质量评估')

print('\n【原始数据缺失值统计】')
missing_raw = raw_df.isnull().sum()
print(missing_raw)

print('\n【清洗后缺失值统计】')
missing_clean = clean_df.isnull().sum()
print(missing_clean)

invalid_phone = clean_df[clean_df['user_phone'] == '无效手机号'].shape[0]
print(f'\n无效手机号数量: {invalid_phone}')

invalid_email = clean_df[clean_df['user_email'] == '无效邮箱'].shape[0]
print(f'无效邮箱数量: {invalid_email}')

invalid_date = clean_df[clean_df['register_date'] == '无效日期'].shape[0]
print(f'无效日期数量: {invalid_date}')

unknown_age = clean_df[clean_df['user_age'] == '未知年龄'].shape[0]
print(f'未知年龄数量: {unknown_age}')

invalid_name = clean_df[clean_df['name_status'] == '姓名异常'].shape[0]
print(f'姓名异常数量: {invalid_name}')

print('\n【运营商分布】')
operator_dist = clean_df['operator_type'].value_counts()
print(operator_dist)

print('\n【年龄分组统计】')
age_group_dist = clean_df['age_group'].value_counts()
print(age_group_dist)

print('\n【省份分布】')
province_dist = clean_df['province'].value_counts()
print(province_dist)
```


=====

数据质量评估

=====

【原始数据缺失值统计】

id	0
raw_name	0
raw_phone	0
raw_email	0
raw_register_time	0
raw_address	0
raw_age	0
raw_remark	0

dtype: int64

【清洗后缺失值统计】

id	0
user_name	0
user_phone	0
user_email	0
register_date	0
user_address	0
user_age	0
user_remark	0
name_status	0
operator_type	0
email_provider	0
age_group	0
province	0

dtype: int64

无效手机号数量: 3
无效邮箱数量: 5
无效日期数量: 1
未知年龄数量: 2
姓名异常数量: 0

【运营商分布】

operator_type	
未知运营商	5
中国移动	3
中国联通	3
中国电信	1

Name: count, dtype: int64

【年龄分组统计】

age_group	
青年	5
中年	5
未知	2

Name: count, dtype: int64

【省份分布】

province	
广东	2
北京	1
上海	1
浙江	1
四川	1

湖北 1
重庆 1
陕西 1
湖南 1
山东 1
云南 1
Name: count, dtype: int64

8. 数据导出

将清洗前后的数据导出为CSV文件，便于后续分析和使用。

```
In [9]: raw_df.to_csv('raw_user_data.csv', index=False, encoding='utf-8-sig')
        clean_df.to_csv('clean_user_data.csv', index=False, encoding='utf-8-sig')

        print('1. raw_user_data.csv - 原始数据')
        print('2. clean_user_data.csv - 清洗后数据')
```

数据已导出：
1. raw_user_data.csv - 原始数据
2. clean_user_data.csv - 清洗后数据

9. 清洗效果分析

清洗前后对比

字段	清洗前问题	清洗后结果
姓名	包含空格、大小写混乱	去除所有空格，统一格式
手机号	包含横杠、字母、86前缀	仅保留11位数字，无效标记
邮箱	大小写混乱、格式错误	小写统一，格式校验
日期	多种格式混搭	统一为YYYY-MM-DD
年龄	包含"岁"字、"未知"	提取纯数字或标记未知
备注	空值、NULL	统一替换为"无备注"

新增分析维度

通过本次清洗，我们还新增了以下分析维度：

- 1. **姓名合法性校验**：识别异常姓名
- 2. **运营商识别**：分析用户手机号所属运营商
- 3. **邮箱服务商**：了解用户偏好的邮箱平台
- 4. **年龄分层**：对用户进行年龄分组
- 5. **地域分布**：提取用户所在省份

10. 关闭数据库连接

完成所有操作后，关闭数据库连接释放资源。

In [10]: `conn.close()`

数据库连接已关闭

=====

数据预处理实习任务完成！

=====

In []: